



1

17

2850 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

Graph and Trees

عملي مشترك



RB Informatics; 20/4/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

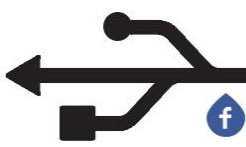
بسم الله الرحمن الرحيم

بعد أن تعلمنا في الفصل الماضي البنى الخطية من Array , Linked List , Queue... واستخدمناها لحل المشاكل التسلسلية البسيطة , يأتي سؤال : ماذا لو أردنا تمثيل شبكة طرق مثل Gps او العلاقات بين الأصدقاء في الفيسبوك أو حتى هيكل مجلدات الحاسوب؟!..

هنا يأتي دور البنى اللاخطية

لذلك سنبدأ اليوم بال graph الذي تمثل الشبكات المتشابهة ثم ننتقل الى حالة خاصة منه وهي ال Trees التي تمثل العلاقات الهرمية و هكذا نكون قد تعرفنا على الأدوات الحقيقية المستخدمة لحل مشاكل العالم الواقعي

أهم العناوين الواردة في المحاضرة	
رقم الصفحة	عنوان الفقرة
2	مفهوم ال Graph
3	مفهوم ال Tree
4	مفهوم ال Binary Tree و أنواعها
7	طرائق تمثيل ال Binary Tree
9	طرائق اجتياز ال Binary Tree
16	تكويد ال Binary Tree و طرائق الاجتياز



What is the graph?

ال graph هو بنى معطيات غير خطية تتكون من مجموعة من ال nodes مرتبطة مع بعضها البعض بـ edges

شرح بعض المصطلحات

Nodes (العقد): تمثل كيانا أو شيئا . مثل : (شخص , مدينة ,)

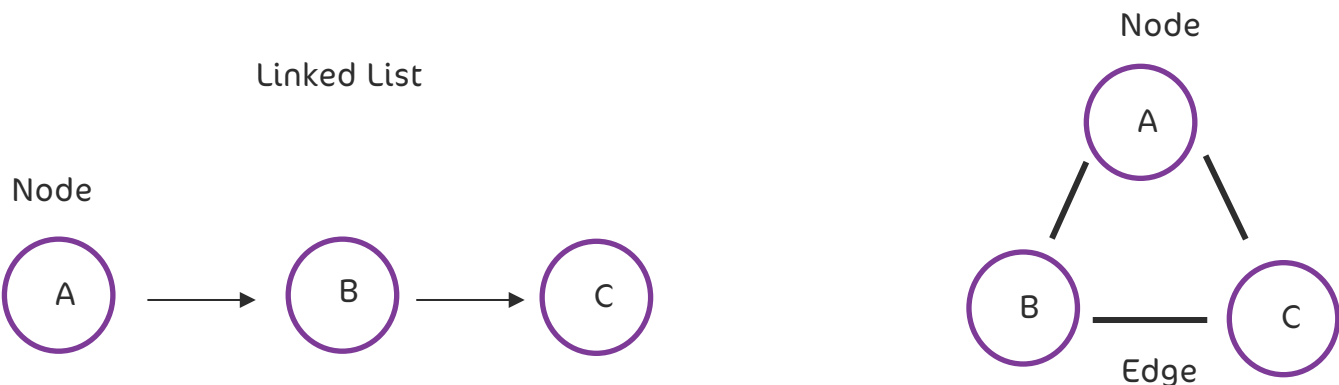
Edges (الحواف): تمثل العلاقة أو الاتصال بين العقد (الصدقة , طريق , رابط تشعبي)

غير خطية : حيث ان ال Nodes ليست مرتبة بشكل تسلسلي (مثل ال Array , او ال List) فإن العقدة يمكن ان تتصل بأي عدد من العقد الأخرى بشكل عشوائي لتشكل هيكل شبكي معقدا.

خطية: مرتبة بشكل تسلسلي (كل عنصر له عنصر سابق وتال باستثناء العنصر الأول والأخير)

■ امثلة عن استخدام ال graph

يمكننا من تمثيل مجمل العلاقات غير الخطية بين البيانات مثلا (شبكات التواصل الاجتماعي , الخرائط...)
وأیضا يمكن استخدامها لتمثيل علاقات خطية كما سبق وفعلنا بال Linked List
(حيث ان ال Linked list هي حالة خاصة من ال graph).



بعد ان تعرفنا على ال graph كبنية معطيات غير خطية تعبر عن علاقات معقدة, نأتي الآن الى النوع الخاص وهو ال Tree

What is Tree ?

هي بنية معطيات هرمية وغير خطية وهي حالة خاصة من ال graph , تتكون من مجموعة من ال Nodes المتصلة مع بعضها ب Edges بشرط ان تكون:

1. غير دورية (Acyclic): لا يوجد مسار يبدأ وينتهي بنفس العقدة
 2. متصلة (connected): أي لا يوجد عقدة او عقد معزولة منفصلة عن باقي العقد
 3. عدد الحواف = عدد العقد - 1
 4. يوجد جذر واحد للشجرة (Root)
 5. كل عقدة عدا الجذر لها اب واحد فقط
- ملاحظة:** لو اختلف أي شرط لم تعد شجرة

ملاحظة اربيسيزية:

في ال graph العقدة قد يكون لها أكثر من أب, أما ال Tree هي graph متصل موجه ولا يحتوي على حلقات cycles وهذا ما يعني ان لكل عقدة أب واحد فقط

المصطلحات الأساسية في ال Trees

- **Root (الجذر):** هي نقطة البداية في الشجرة وهي العقدة الوحيدة التي ليس لديها اب
- **Child (الابن):** العقدة التي تقع اسفل عقدة أخرى وتتصل بها مباشرة
- **Parent (الاب):** العقدة التي تقع فوق عقدة أخرى وتتصل بها مباشرة
- **Sibling :** عقدتان أو أكثر تشتركان في نفس الأب
- **Leaf او External Node :** العقدة التي ليس لها أبناء
- **Internal Node :** العقدة التي لديها ابن واحد على الأقل (ليست ورقة)
- **Tree level :** عدد طبقات الشجرة
- **درجة العقدة:** عدد أبناء العقدة
- **Subtree (الشجرة الفرعية):** أي عقدة مع جميع احفادها تشكل شجرة فرعية



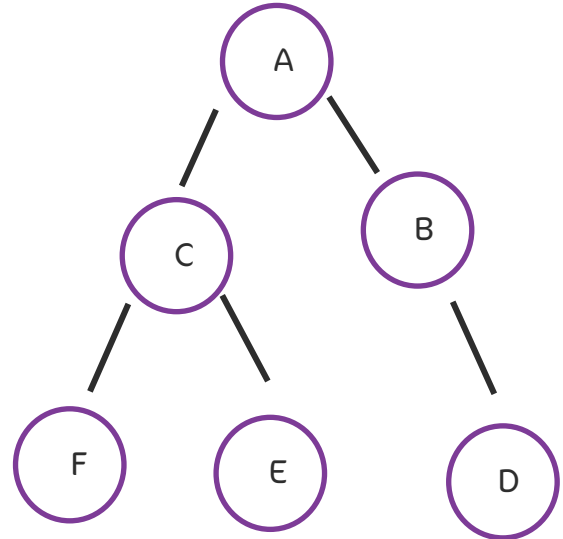
LEVEL 1



LEVEL 2



LEVEL 3



- هذه شجرة من المستوى الثالث
- الجذر هو A ,العقد الداخلية B و C , العقد الخارجية D E F

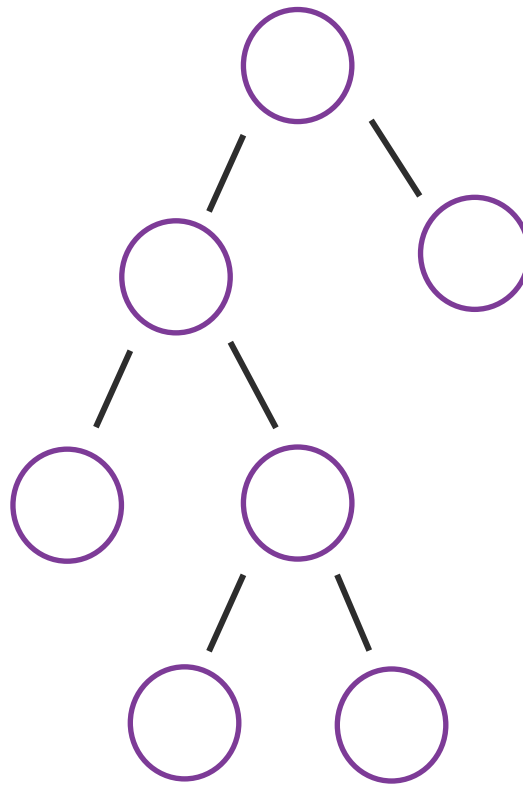
Binary Tree

الآن سنتحدث عن نوع محدد أكثر من الأشجار وتمثيله برمجيا وهو ال Binary Tree حيث سنقوم بوضع قيد إضافي على ال Tree وهو درجة كل العقد 2 على الأكثر (أي درجة الشجرة 2 على الأكثر) حيث كل عقدة تملك ابن او ابنين او لا ابن وهما: Left Child و Right Child .

بعض أنواع ال Binary Tree

1. Full Binary Tree:

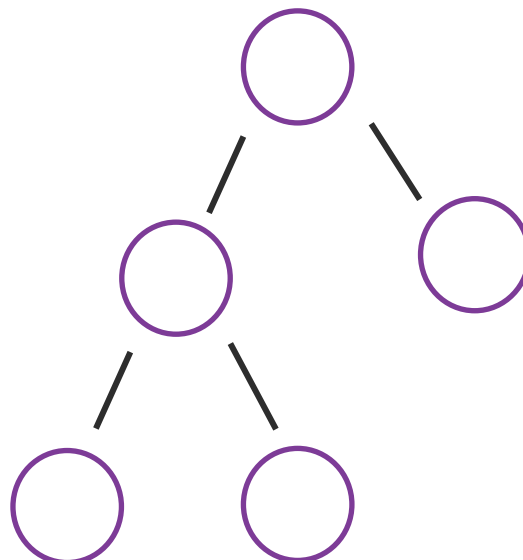
كل عقدة من الشجرة إما أن تملك ابنان فقط أو أن تكون ورقة (leaf) (بدون أبناء)



2. Complete Binary Tree:

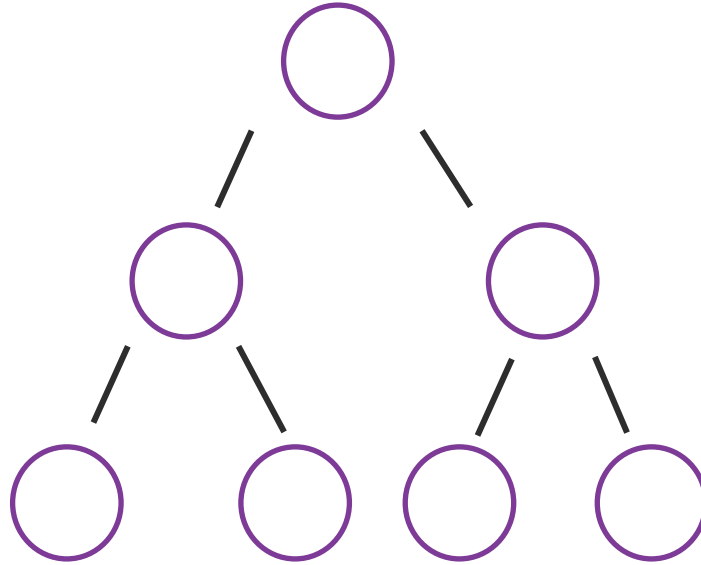
جميع المستويات ممتلئة بالكامل ما عدا المستوى الأخير يمكن ان يحتوي على فراغات ولكن يشترط ان تكون من اليمين

باتجاه اليسار حصرا



3. Perfect Binary Tree:

جميع العقد الداخلية لها اثنان وجميع الأوراق في مستوى واحد

ملاحظات:

- كل perfect هو full و complete والعكس غير صحيح
- العدد الأقصى للعقد في المستوى الواحد هو $2^{level-1}$ حيث نبدأ ال level من 1
- أقصى عدد من العقد في الشجرة هو $2^{level} - 1$ حيث ال level هنا هو عدد المستويات (مستوى الشجرة) او $2^{h+1} - 1$ (حيث h هو ارتفاع الشجرة)

بعض استخدامات الأشجار Trees :

تستخدم الأشجار في تمثيل التسلسل الهرمي لتخزين المعلومات التي تشكل بطبيعتها تسلسل هرميا كالمجلدات و الملفات او هيكل اداري في شركة او شجرة عائلة

Binary Tree Representation:

يمكن تمثيلها بطريقتين:

1. Array based Representation:

- تحتاج الى مصفوفة أحادية البعد
- أقصى حجم للمصفوفة هو $2^n - 1$ حيث n هو عمق الشجرة depth (عدد المستويات)
- نبدأ بتخزين الجذر (Root) بال index 1 من المصفوفة , حيث نترك الموقع 0 فارغا (لتسهيل العمليات الحسابية) ثم نجد ان لكل element في ال index i

- Left child:

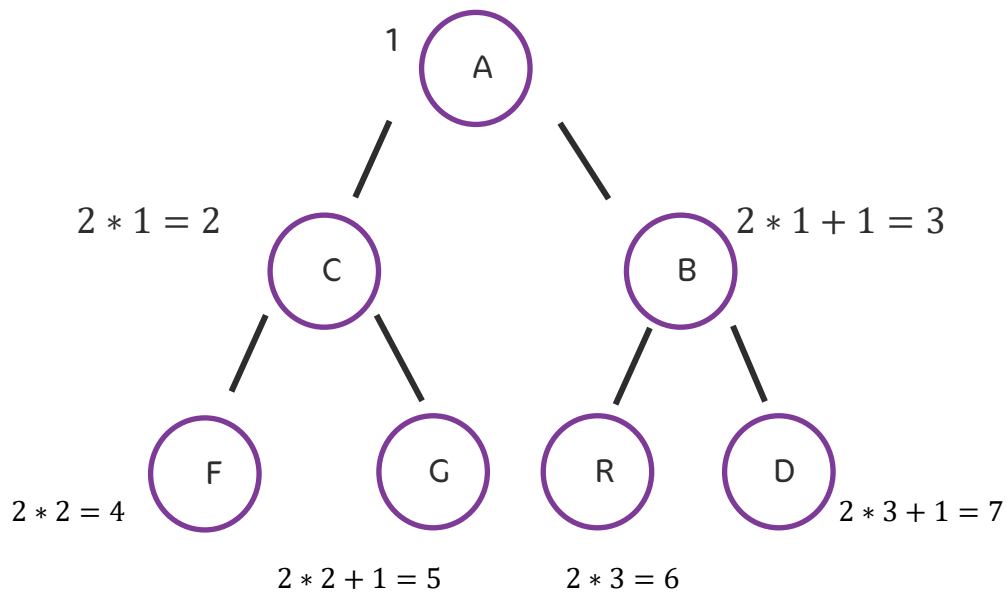
موجود في ال index $2i$

- Right child:

موجود في ال index $2i + 1$

: Ex

لتكن لدينا الشجرة التالية:

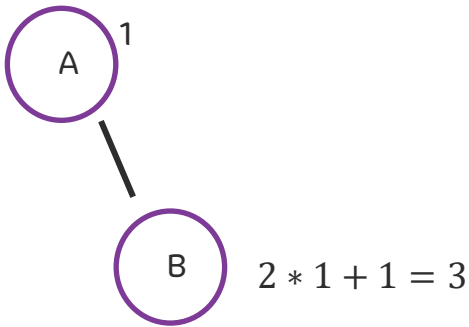


فإن المصفوفة الموافقة لها هي

	A	C	B	F	G	R	D
0	1	2	3	4	5	6	7

ماذا لو كانت الشجرة غير مكتملة (تحتوي فراغات):

EX :



	A		B
0	1	2	3

الموقع 2 يبقى فارغا , لان العقدة 1 ليس بها ابن ايسر

عيوب تمثيل المصفوفة:

- هدر كبير في الذاكرة اذا كانت الشجرة غير مكتملة
- يوجد حد اقصى للحجم لا يمكن تجاوزه

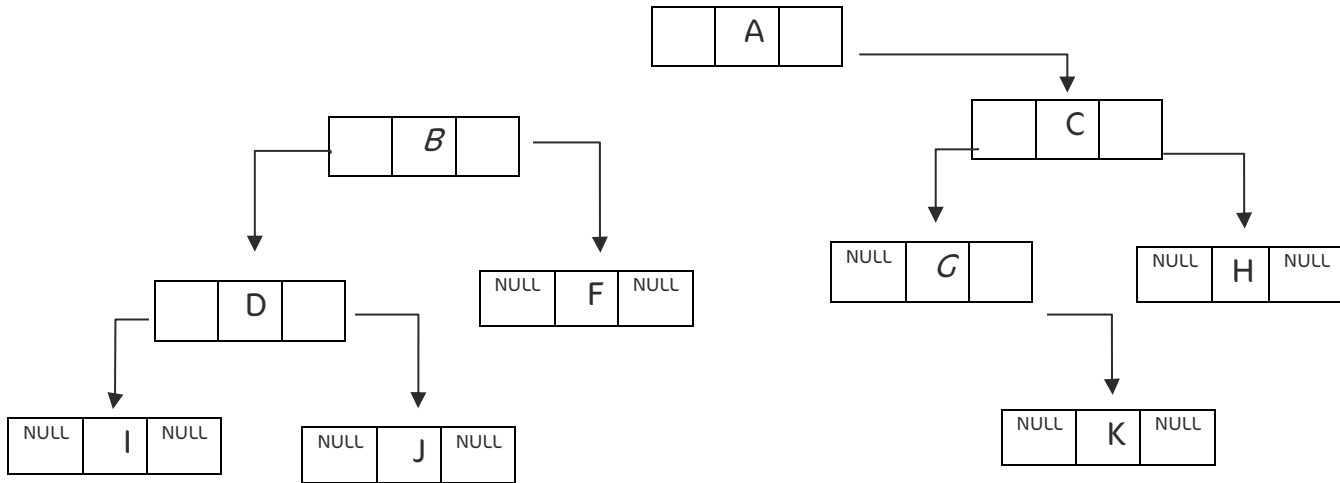
2. Linked List based Representation:

في هذا التمثيل كل عقدة في الشجرة تصمم ككيان مستقل يحتوي على ثلاثة حقول :

1. البيانات Data
2. مؤشر الى الابن الايسر Left child Address
3. مؤشر الى الابن الايمن Right child Address

- اذا لم يكن للعقدة ابن ايسر او ايمن يشير المؤشر الى Null
- ترتبط العقد معا عبر هذه المؤشرات لتشكل بنية الشجرة

مثال توضيحي:



ميزة هذا التمثيل: لا يوجد هدر في الذاكرة فنحن ننشئ عقدة فقط عند الحاجة اليها وليس هناك حد اقصى لعدد ال Nodes , أما عيبه فهو الوصول الى عقدة معينة يحتاج اجتياز المؤشرات من الجذر بينما في المصفوفة يكون الوصول مباشرا

BT Traversal (Binary Tree Traversal)

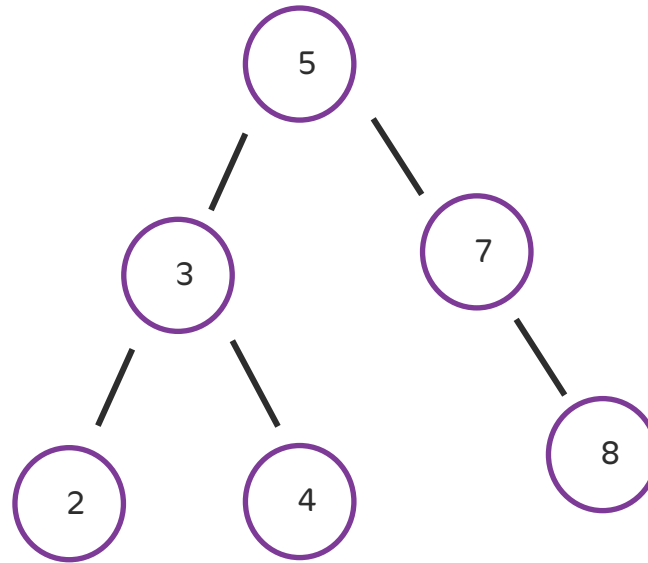
اجتياز الشجرة الثنائية : ويعني زيارة جميع العقد في الشجرة مرة واحدة وبترتيب معين.

وهناك طريقتان رئيستان للاجتياز:

1.BFS(Breadth First Search)

او ما يعرف ب level order traversal

الفكرة: نزور العقد مستوى مستوى (level by level) حيث نبدأ من الجذر (المستوى الاول) ثم نزور كل العقد في المستوى الثاني من اليسار الى اليمين وهكذا وصولا الى آخر مستوى



BFS: 5_3_7_2_4_8

2.DFS (Depth First Search)

الفكرة: نبدأ من الجذر ونتعمق قدر الإمكان في الفرع الأيسر (أو حسب الترتيب) قبل العودة إلى اليمين ، ينفذ عادة باستخدام الاستدعاء الذاتي (Recursion)
وله ثلاثة أنواع رئيسية:

أولاً : In_Order Traversal :

Left → Root → Right

الخطوات:

(1) اطبع الابن الأيسر

(2) اطبع الجذر

(3) اطبع الابن الأيمن

(طبعاً يمكن أن نجري أي عملية غير الطباعة)

• في (BST) ال In_Order يعطي الأرقام مرتبة تصاعدياً

ثانيا : Preorder Traversal

Root → Left → Right

الخطوات:

- (1) اطبع الجذر
 - (2) اطبع الابن الايسر
 - (3) اطبع الابن الأيمن
- يستخدم في نسخ الشجرة (Tree Copying) او تصديرها بحيث يمكن إعادة بنائها لاحقا

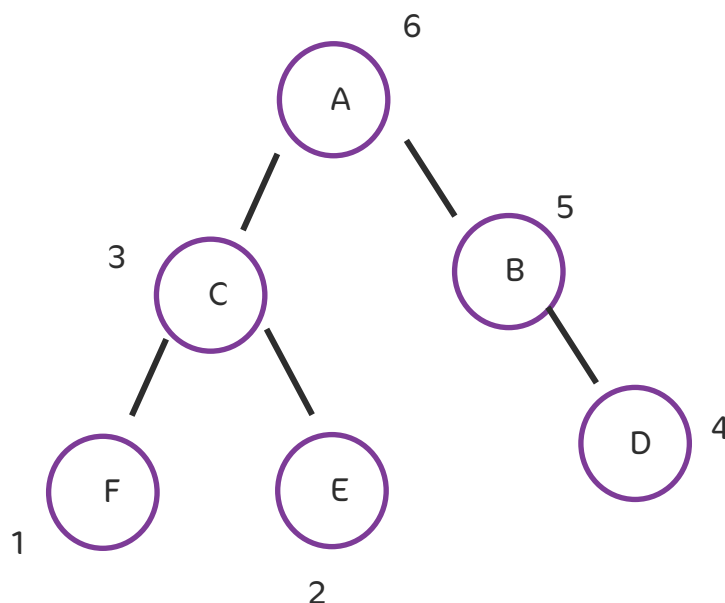
ثالثا : Post_Order Traversal :

Left → Right → Root

الخطوات:

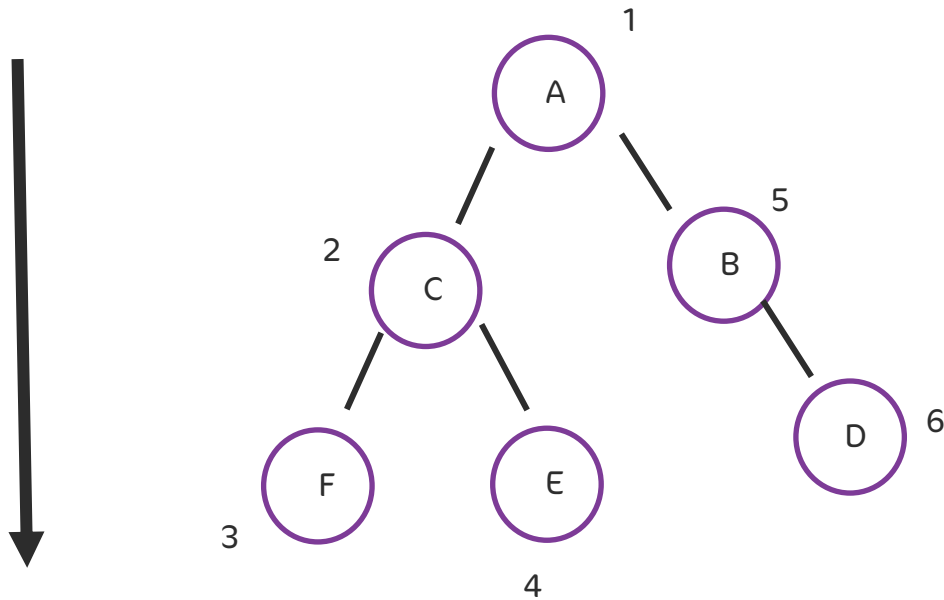
- (1) اطبع الابن الايسر
 - (2) اطبع الابن الأيمن
 - (3) اطبع الجذر
- يستخدم في حذف الشجرة (Tree deletion) لأننا نحذف الأبناء قبل الاب(والا سنفقد القدرة على الوصول الى الابناء).
- هذا على مستوى شجرة صغيرة فيها جذر و ابنان فقط ويمكن تعميم هذا النمط على الاشجار الأكبر فنقوم باستدعاء عودي على الأبناء حتى نصل الى الأوراق ثم نعود ادراجنا ونعمم احد الأنماط السابقة

1. Post_Order Traversal :

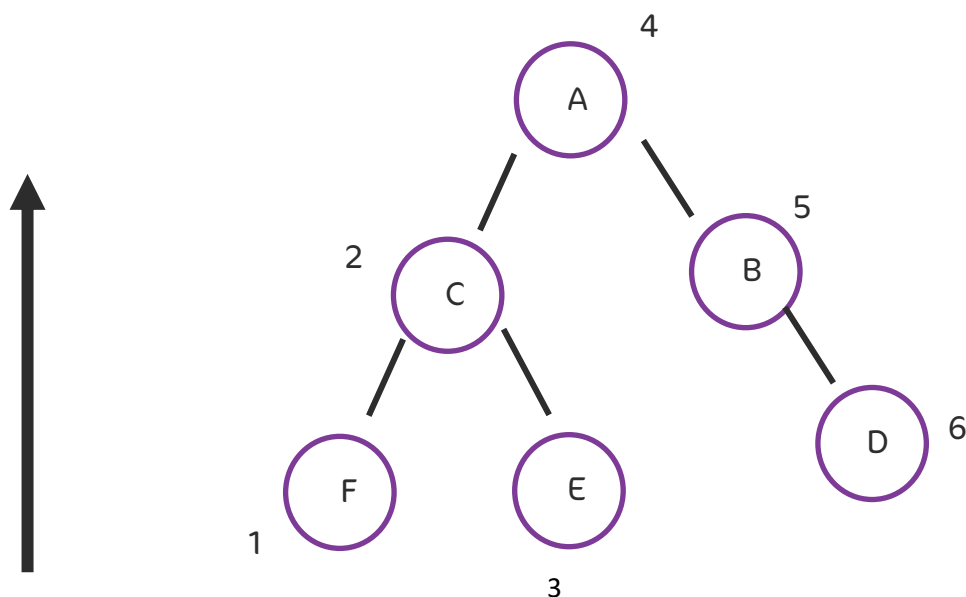




2. Pre_Order Traversal :



3. In_Order Traversal :



والآن سنكتب الخوارزميات الثلاثة باستخدام ال pseudocode مع الشرح (حيث بآخر المحاضرة سنكودها بلغة جافا)

Post_Order Traversal :

POSTORDER-TREE-WALK(x)

- 1) If $x \neq \text{NIL}$
- 2) POSTORDER-TREE-WALK(x.left)
- 3) POSTORDER-TREE-WALK(x.right)
- 4) Print x.key

شرح الاسطر:

السطر 1_ اذا كانت العقدة ليست فارغة نفذ (حيث هذا ال "Basecase")
السطر 2_ اذهب أولا الى الابن الايسر بشكل متكرر وتعمق فيه قدر الإمكان
السطر 3_ ثم ارجع واذهب الى الابن الأيمن بشكل متكرر وتعمق فيه قدر لامكان
السطر 4_ بعد الانتهاء من الأبناء اطبع قيمة العقدة الحالية

Pre_Order Traversal :

PREORDER-TREE-WALK(x)

- 1- If $x \neq \text{NIL}$
- 2- Print x.key
- 3- PREORDER-TREE-WALK(x. left)
- 4- PREORDER-TREE-WALK(x.right)

نفس الشرح السابق ولكن بغير ترتيب

In_Order Traversal :

- 1- if $x \neq \text{NIL}$
- 2- INORDER-TREE-WALK(x.left)
- 3- Print x.key
- 4- INORDER-TREE-WALK(x.right)

سؤال:

لدينا Binary Tree تم اجتيازها بطريقتين

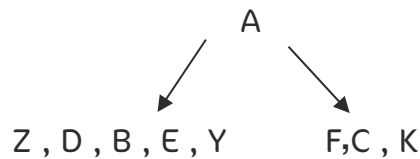
- In_Ordre Traversal:
Z , D , B , E , Y , A , F , C , K
- Pre_Order Traversal:
A , B , D , Z , E , Y , C , F , K

المطلوب استنتاج شكل الشجرة الأصلي و ارسما

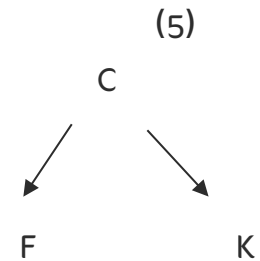
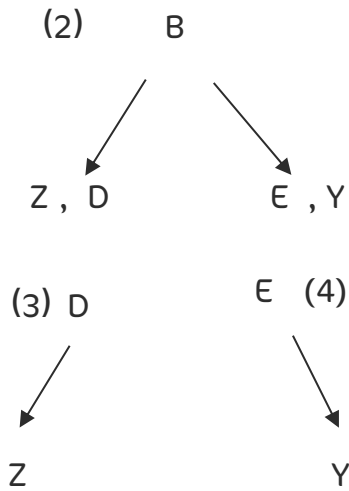
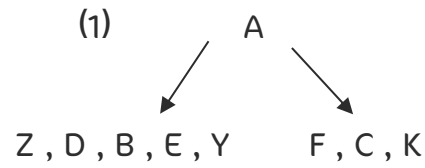
خطوات الحل:

- 1- اول عنصر في ال Pre_Order = جذر الشجرة (او جذر أي جزء منها)
 - 2- ابحث عن هذا الجذر في In_Order ← كل ما قبله يسار وكل ما بعده يمين
 - 3- لكل جذر جديد (يسار او يمين) ارجع الى Pre-Order وخذ اول عنصر يظهر من هذا الجزء = جذر جديد
 - 4- كرر الخطوات حتى تنتهي جميع الأجزاء
- نلاحظ ان اول عنصر في ال pre_order هو جذر الشجرة root = A
نبحث عنه في ال In_order ← كل ما قبله يسار وكل ما بعده يمين

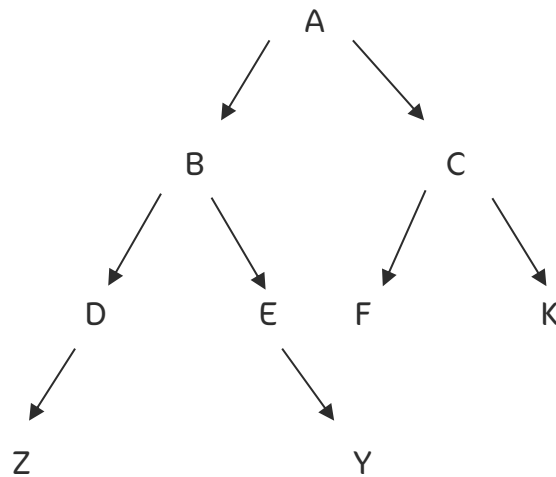
• الرسم للتوضيح :



وهكذا في كل مرة نأخذ عنصر تلو عنصر بالترتيب من ال Pre_Order وتنقسم العناصر في ال In_Order بهذا العنصر حيث يعتبر Root لشجرة فرعية فيه



نركب الشجرة و نحصل على الحل النهائي :



للتأكد من الحل أعد كتابة ال Pre_Order و ال In_Order من هذه الشجرة يجب ان يماثل المعطيات السابقة

"CODE"

تحميل ال Binary Tree باستخدام ال Array

```
1 class BinaryTree{
2     // المصفوفة التي تحمل عقد هذه الشجرة
3     char []arr;
4
5     // باني نمرر له المصفوفة التي تمثل الشجرة
6     BinaryTree(char []arr){
7         this.arr = arr;
8     }
9
10    public void preOrderTraversal (int i){
11        // شرط التوقف: اذا وجدت المكان فارغا أو خرجنا خارج إطار المصفوفة قم بالتوقف
12        if (i >= arr.length||arr[i]== ' ')
13            return ;
14        // نطبع الجذر قبل الانطلاق إلى الأبناء
15        System.out.print(arr[i]);
16        // استدعاء التابع العودي على الابن الأيسر
17        preOrderTraversal(2*i);
18        // استدعاء التابع العودي على الابن الأيمن
19        preOrderTraversal(2*i+1);
20    }
21
22    public void postOrderTraversal (int i){
23        // شرط التوقف: اذا وجدت المكان فارغا أو خرجنا خارج إطار المصفوفة قم بالتوقف
24        if (i >= arr.length||arr[i]== ' ')
25            return ;
26        // استدعاء التابع العودي على الابن الأيسر
27        postOrderTraversal(2*i);
28        // استدعاء التابع العودي على الابن الأيمن
29        postOrderTraversal(2*i+1);
30        // بعد الانتهاء من الاستدعاءات على كل الابنين نطبع الجذر
31        System.out.print(arr[i]);
32    }
```

```
1 public void inOrderTraversal (int i) {
2     // // التوقف: اذا وجدت المكان فارغا أو خرجنا خارج إطار المصفوفة قم بالتوقف
3     if (i >= arr.length || arr[i] == ' ')
4         return;
5
6     // استدعاء التابع العودي على الابن الأيسر
7     inOrderTraversal(2*i);
8
9     // طباعة الجذر بعد انتهاء الاستدعاء العودي على الابن الأيسر
10    System.out.print(arr[i]);
11
12    // استدعاء التابع العودي على الابن الأيمن
13    inOrderTraversal(2*i+1);
14 }
15 }
16 public class Main {
17     public static void main(String[] args) {
18         // سنملأ العقد الغير موجودة بمحرف SPACE و نترك أول عنصر بالمصفوفة فارغاً كذلك
19         char []arr = {' ', 'A', 'C', 'B', 'F', 'G', ' ', 'D'};
20
21         // أنشأنا شجرة بناء على هذه المصفوفة
22         BinaryTree bt = new BinaryTree(arr);
23
24         // preorder traversal starting from the root
25         bt.preOrderTraversal(1);
26
27         System.out.println();
28         // inorder traversal starting from the root
29         bt.inOrderTraversal(1);
30
31         System.out.println();
32         // postorder traversal starting from the root
33         bt.postOrderTraversal(1);
34
35         System.out.println();
36     }}
```

تمثيل ال Binary Tree باستخدام ال Linked List

```
2 class BinaryTree{
3     // جذر الشجرة
4     Node root;
5     // كلاس داخلي لتمثيل العقد
6     class Node {
7         char value;
8         Node left, right;
9
10        // بالوضع الطبيعي عندما ننشئ عقدة جديدة نسند لها قيمة و لا يكون لها أبناء
11        Node(char value) {
12            this.value = value;
13            left = right = null;
14        }
15    }
16
17    نفس المبدأ السابق تماماً لكن عوضاً عن الاستدعاء العودي على indexes ستقوم بالاستدعاء العودي على الأبناء كعقد فعلية
18
19    وشرط التوقف يصبح أن تكون العقدة التي وصلنا لها null أما سابقاً فكنا نتوقف إن خرجنا خارج إطار المصفوفة أو وجدنا محرف space ضمن المصفوفة
20
21    18 public void preOrderTraversal(Node node) {
22        if (node == null)
23            return;
24        System.out.print(node.value);
25        preOrderTraversal(node.left);
26        preOrderTraversal(node.right);
27    }
```



```
1 public void inOrderTraversal(Node node) {
2     if (node == null)
3         return;
4     inOrderTraversal(node.left);
5     System.out.print(node.value);
6     inOrderTraversal(node.right);
7 }
8
9 public void postOrderTraversal(Node node) {
10    if (node == null)
11        return;
12    postOrderTraversal(node.left);
13    postOrderTraversal(node.right);
14    System.out.print(node.value);
15 }}
```



عندما لا تجد الطريق
المؤدي الى النجاح
سيكون عليك ان
تبتكره

```
1 public class Main {
2     public static void main(String[] args) {
3         BinaryTree bt = new BinaryTree();
4         bt.root = bt.new Node('A');
5         bt.root.left = bt.new Node('C');
6         bt.root.left.left = bt.new Node('F');
7         bt.root.left.right = bt.new Node('G');
8         bt.root.right = bt.new Node('B');
9         bt.root.right.right = bt.new Node('D');
10    }
11    // نستدعي التتابع على الجذر
12    bt.preOrderTraversal(bt.root);
13    System.out.println();
14    bt.inOrderTraversal(bt.root);
15    System.out.println();
16    bt.postOrderTraversal(bt.root);
17 }}
```